Configuration

Resources that Kubernetes provides for configuring Pods.

1 - ConfigMaps

A ConfigMap is an API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume.

A ConfigMap allows you to decouple environment-specific configuration from your container images, so that your applications are easily portable.

> **Caution:**
>
> ConfigMap does not provide secrecy or encryption. If the data you want to store are confidential, use a Secret rather than a ConfigMap, or use additional (third party) tools to keep your data private.

Motivation

Use a ConfigMap for setting configuration data separately from application code.

For example, imagine that you are developing an application that you can run on your own computer (for development) and in the cloud (to handle real traffic). You write the code to look in an environment variable named `DATABASE_HOST`. Locally, you set that variable to `localhost`. In the cloud, you set it to refer to a Kubernetes Service that exposes the database component to your cluster. This lets you fetch a container image running in the cloud and debug the exact same code locally if needed.

> **Note:**
> A ConfigMap is not designed to hold large chunks of data. The data stored in a ConfigMap cannot exceed 1 MiB. If you need to store settings that are larger than this limit, you may want to consider mounting a volume or use a separate database or file service.

ConfigMap object

A ConfigMap is an API object that lets you store configuration for other objects to use. Unlike most Kubernetes objects that have a `spec`, a ConfigMap has `data` and `binaryData` fields. These fields accept key-value pairs as their values. Both the `data` field and the `binaryData` are optional. The `data` field is designed to contain UTF-8 strings while the `binaryData` field is designed to contain binary data as base64-encoded strings.

The name of a ConfigMap must be a valid DNS subdomain name.

Each key under the `data` or the `binaryData` field must consist of alphanumeric characters, `-`, `_` or `.`. The keys stored in `data` must not overlap with the keys in the `binaryData` field.

Starting from v1.19, you can add an `immutable` field to a ConfigMap definition to create an immutable ConfigMap.

ConfigMaps and Pods

You can write a Pod `spec` that refers to a ConfigMap and configures the container(s) in that Pod based on the data in the ConfigMap. The Pod and the ConfigMap must be in the same namespace.

> **Note:**
> The `spec` of a static Pod cannot refer to a ConfigMap or any other API objects.

Here's an example ConfigMap that has some keys with single values, and other keys where the value looks like a fragment of a configuration format.

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  # property-like keys; each key maps to a simple value
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"

  # file-like keys
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
  user-interface.properties: |
    color.good=purple
    color.bad=yellow
    allow.textmode=true
```

There are four different ways that you can use a ConfigMap to configure a container inside a Pod:

1. Inside a container command and args
2. Environment variables for a container
3. Add a file in read-only volume, for the application to read
4. Write code to run inside the Pod that uses the Kubernetes API to read a ConfigMap

These different methods lend themselves to different ways of modeling the data being consumed. For the first three methods, the kubelet uses the data from the ConfigMap when it launches container(s) for a Pod.

The fourth method means you have to write code to read the ConfigMap and its data. However, because you're using the Kubernetes API directly, your application can subscribe to get updates whenever the ConfigMap changes, and react when that happens. By accessing the Kubernetes API directly, this technique also lets you access a ConfigMap in a different namespace.

Here's an example Pod that uses values from `game-demo` to configure a Pod:

configmap/configure-pod.yaml 

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-demo-pod
spec:
  containers:
    - name: demo
      image: alpine
      command: ["sleep", "3600"]
      env:
        # Define the environment variable
        - name: PLAYER_INITIAL_LIVES # Notice that the case is different here
                                     # from the key name in the ConfigMap.
          valueFrom:
            configMapKeyRef:
              name: game-demo           # The ConfigMap this value comes from.
              key: player_initial_lives # The key to fetch.
        - name: UI_PROPERTIES_FILE_NAME
          valueFrom:
            configMapKeyRef:
              name: game-demo
              key: ui_properties_file_name
      volumeMounts:
      - name: config
        mountPath: "/config"
        readOnly: true
  volumes:
  # You set volumes at the Pod level, then mount them into containers inside that Pod
  - name: config
    configMap:
      # Provide the name of the ConfigMap you want to mount.
      name: game-demo
      # An array of keys from the ConfigMap to create as files
      items:
      - key: "game.properties"
        path: "game.properties"
      - key: "user-interface.properties"
        path: "user-interface.properties"
```

A ConfigMap doesn't differentiate between single line property values and multi-line file-like values. What matters is how Pods and other objects consume those values.

For this example, defining a volume and mounting it inside the `demo` container as `/config` creates two files, `/config/game.properties` and `/config/user-interface.properties`, even though there are four keys in the ConfigMap. This is because the Pod definition specifies an `items` array in the `volumes` section. If you omit the `items` array entirely, every key in the ConfigMap becomes a file with the same name as the key, and you get 4 files.

Using ConfigMaps

ConfigMaps can be mounted as data volumes. ConfigMaps can also be used by other parts of the system, without being directly exposed to the Pod. For example, ConfigMaps can hold data that other parts of the system should use for configuration.

The most common way to use ConfigMaps is to configure settings for containers running in a Pod in the same namespace. You can also use a ConfigMap separately.

For example, you might encounter addons or operators that adjust their behavior based on a ConfigMap.

Using ConfigMaps as files from a Pod

To consume a ConfigMap in a volume in a Pod:

1. Create a ConfigMap or use an existing one. Multiple Pods can reference the same ConfigMap.
2. Modify your Pod definition to add a volume under `.spec.volumes[]`. Name the volume anything, and have a `.spec.volumes[].configMap.name` field set to reference your ConfigMap object.
3. Add a `.spec.containers[].volumeMounts[]` to each container that needs the ConfigMap. Specify `.spec.containers[].volumeMounts[].readOnly = true` and `.spec.containers[].volumeMounts[].mountPath` to an unused directory name where you would like the ConfigMap to appear.
4. Modify your image or command line so that the program looks for files in that directory. Each key in the ConfigMap `data` map becomes the filename under `mountPath`.

This is an example of a Pod that mounts a ConfigMap in a volume:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    configMap:
      name: myconfigmap
```

Each ConfigMap you want to use needs to be referred to in `.spec.volumes`.

If there are multiple containers in the Pod, then each container needs its own `volumeMounts` block, but only one `.spec.volumes` is needed per ConfigMap.

Mounted ConfigMaps are updated automatically

When a ConfigMap currently consumed in a volume is updated, projected keys are eventually updated as well. The kubelet checks whether the mounted ConfigMap is fresh on every periodic sync. However, the kubelet uses its local cache for getting the current value of the ConfigMap. The type of the cache is configurable using the `configMapAndSecretChangeDetectionStrategy` field in the KubeletConfiguration struct. A ConfigMap can be either propagated by watch (default), ttl-based, or by redirecting all requests directly to the API server. As a result, the total delay from the moment when the ConfigMap is updated to the moment when new keys are projected to the Pod can be as long as the kubelet sync period + cache propagation delay, where the cache propagation delay depends on the chosen cache type (it equals to watch propagation delay, ttl of cache, or zero correspondingly).

ConfigMaps consumed as environment variables are not updated automatically and require a pod restart.

> **Note:**
> A container using a ConfigMap as a subPath volume mount will not receive ConfigMap updates.

Using Configmaps as environment variables

To use a Configmap in an environment variable in a Pod:

1. For each container in your Pod specification, add an environment variable for each Configmap key that you want to use to the `env[].valueFrom.configMapKeyRef` field.
2. Modify your image and/or command line so that the program looks for values in the specified environment variables.

This is an example of defining a ConfigMap as a pod environment variable:

The following ConfigMap (myconfigmap.yaml) stores two properties: username and access_level:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: myconfigmap
data:
  username: k8s-admin
  access_level: "1"
```

The following command will create the ConfigMap object:

```
kubectl apply -f myconfigmap.yaml
```

The following Pod consumes the content of the ConfigMap as environment variables:

configmap/env-configmap.yaml

```yaml
apiVersion: v1
kind: Pod
```

```
metadata:
  name: env-configmap
spec:
  containers:
    - name: app
      command: ["/bin/sh", "-c", "printenv"]
      image: busybox:latest
      envFrom:
        - configMapRef:
            name: myconfigmap
```

The `envFrom` field instructs Kubernetes to create environment variables from the sources nested within it. The inner `configMapRef` refers to a ConfigMap by its name and selects all its key-value pairs. Add the Pod to your cluster, then retrieve its logs to see the output from the printenv command. This should confirm that the two key-value pairs from the ConfigMap have been set as environment variables:

```
kubectl apply -f env-configmap.yaml
```

```
kubectl logs pod/env-configmap
```

The output is similar to this:

```
...
username: "k8s-admin"
access_level: "1"
...
```

Sometimes a Pod won't require access to all the values in a ConfigMap. For example, you could have another Pod which only uses the username value from the ConfigMap. For this use case, you can use the `env.valueFrom` syntax instead, which lets you select individual keys in a ConfigMap. The name of the environment variable can also be different from the key within the ConfigMap. For example:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: env-configmap
spec:
  containers:
  - name: envars-test-container
    image: nginx
    env:
    - name: CONFIGMAP_USERNAME
      valueFrom:
        configMapKeyRef:
          name: myconfigmap
          key: username
```

In the Pod created from this manifest, you will see that the environment variable `CONFIGMAP_USERNAME` is set to the value of the `username` value from the ConfigMap. Other keys from the ConfigMap data are not copied into the environment.

It's important to note that the range of characters allowed for environment variable names in pods is restricted. If any keys do not meet the rules, those keys are not made available to your container, though the Pod is allowed to start.

Immutable ConfigMaps

ⓘ **FEATURE STATE:** Kubernetes v1.21 [stable]

The Kubernetes feature *Immutable Secrets and ConfigMaps* provides an option to set individual Secrets and ConfigMaps as immutable. For clusters that extensively use ConfigMaps (at least tens of thousands of unique ConfigMap to Pod mounts), preventing changes to their data has the following advantages:

- protects you from accidental (or unwanted) updates that could cause applications outages
- improves performance of your cluster by significantly reducing load on kube-apiserver, by closing watches for ConfigMaps marked as immutable.

You can create an immutable ConfigMap by setting the `immutable` field to `true`. For example:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  ...
data:
  ...
immutable: true
```

Once a ConfigMap is marked as immutable, it is *not* possible to revert this change nor to mutate the contents of the `data` or the `binaryData` field. You can only delete and recreate the ConfigMap. Because existing Pods maintain a mount point to the deleted ConfigMap, it is recommended to recreate these pods.

What's next

- Read about Secrets.
- Read Configure a Pod to Use a ConfigMap.
- Read about changing a ConfigMap (or any other Kubernetes object)
- Read The Twelve-Factor App to understand the motivation for separating code from configuration.

2 - Secrets

A Secret is an object that contains a small amount of sensitive data such as a password, a token, or a key. Such information might otherwise be put in a Pod specification or in a container image. Using a Secret means that you don't need to include confidential data in your application code.

Because Secrets can be created independently of the Pods that use them, there is less risk of the Secret (and its data) being exposed during the workflow of creating, viewing, and editing Pods. Kubernetes, and applications that run in your cluster, can also take additional precautions with Secrets, such as avoiding writing sensitive data to nonvolatile storage.

Secrets are similar to ConfigMaps but are specifically intended to hold confidential data.

> **Caution:**
>
> Kubernetes Secrets are, by default, stored unencrypted in the API server's underlying data store (etcd). Anyone with API access can retrieve or modify a Secret, and so can anyone with access to etcd. Additionally, anyone who is authorized to create a Pod in a namespace can use that access to read any Secret in that namespace; this includes indirect access such as the ability to create a Deployment.
>
> In order to safely use Secrets, take at least the following steps:
>
> 1. Enable Encryption at Rest for Secrets.
> 2. Enable or configure RBAC rules with least-privilege access to Secrets.
> 3. Restrict Secret access to specific containers.
> 4. Consider using external Secret store providers.
>
> For more guidelines to manage and improve the security of your Secrets, refer to Good practices for Kubernetes Secrets.

See Information security for Secrets for more details.

Uses for Secrets

You can use Secrets for purposes such as the following:

- Set environment variables for a container.
- Provide credentials such as SSH keys or passwords to Pods.
- Allow the kubelet to pull container images from private registries.

The Kubernetes control plane also uses Secrets; for example, bootstrap token Secrets are a mechanism to help automate node registration.

Use case: dotfiles in a secret volume

You can make your data "hidden" by defining a key that begins with a dot. This key represents a dotfile or "hidden" file. For example, when the following Secret is mounted into a volume, `secret-volume`, the volume will contain a single file, called `.secret-file`, and the `dotfile-test-container` will have this file present at the path `/etc/secret-volume/.secret-file`.

> **Note:**
> Files beginning with dot characters are hidden from the output of `ls -l`; you must use `ls -la` to see them when listing directory contents.

secret/dotfile-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: dotfile-secret
data:
  .secret-file: dmFsdWUtMg0KDQo=
---
apiVersion: v1
kind: Pod
metadata:
  name: secret-dotfiles-pod
spec:
  volumes:
    - name: secret-volume
      secret:
        secretName: dotfile-secret
  containers:
    - name: dotfile-test-container
      image: registry.k8s.io/busybox
```

```
      command:
        - ls
        - "-l"
        - "/etc/secret-volume"
      volumeMounts:
        - name: secret-volume
          readOnly: true
          mountPath: "/etc/secret-volume"
```

Use case: Secret visible to one container in a Pod

Consider a program that needs to handle HTTP requests, do some complex business logic, and then sign some messages with an HMAC. Because it has complex application logic, there might be an unnoticed remote file reading exploit in the server, which could expose the private key to an attacker.

This could be divided into two processes in two containers: a frontend container which handles user interaction and business logic, but which cannot see the private key; and a signer container that can see the private key, and responds to simple signing requests from the frontend (for example, over localhost networking).

With this partitioned approach, an attacker now has to trick the application server into doing something rather arbitrary, which may be harder than getting it to read a file.

Alternatives to Secrets

Rather than using a Secret to protect confidential data, you can pick from alternatives.

Here are some of your options:

- If your cloud-native component needs to authenticate to another application that you know is running within the same Kubernetes cluster, you can use a ServiceAccount and its tokens to identify your client.
- There are third-party tools that you can run, either within or outside your cluster, that manage sensitive data. For example, a service that Pods access over HTTPS, that reveals a Secret if the client correctly authenticates (for example, with a ServiceAccount token).
- For authentication, you can implement a custom signer for X.509 certificates, and use CertificateSigningRequests to let that custom signer issue certificates to Pods that need them.

- You can use a device plugin to expose node-local encryption hardware to a specific Pod. For example, you can schedule trusted Pods onto nodes that provide a Trusted Platform Module, configured out-of-band.

You can also combine two or more of those options, including the option to use Secret objects themselves.

For example: implement (or deploy) an operator that fetches short-lived session tokens from an external service, and then creates Secrets based on those short-lived session tokens. Pods running in your cluster can make use of the session tokens, and operator ensures they are valid. This separation means that you can run Pods that are unaware of the exact mechanisms for issuing and refreshing those session tokens.

Types of Secret

When creating a Secret, you can specify its type using the `type` field of the Secret resource, or certain equivalent `kubectl` command line flags (if available). The Secret type is used to facilitate programmatic handling of the Secret data.

Kubernetes provides several built-in types for some common usage scenarios. These types vary in terms of the validations performed and the constraints Kubernetes imposes on them.

Built-in Type	Usage
`Opaque`	arbitrary user-defined data
`kubernetes.io/service-account-token`	ServiceAccount token
`kubernetes.io/dockercfg`	serialized `~/.dockercfg` file
`kubernetes.io/dockerconfigjson`	serialized `~/.docker/config.json` file
`kubernetes.io/basic-auth`	credentials for basic authentication
`kubernetes.io/ssh-auth`	credentials for SSH authentication
`kubernetes.io/tls`	data for a TLS client or server
`bootstrap.kubernetes.io/token`	bootstrap token data

You can define and use your own Secret type by assigning a non-empty string as the `type` value for a Secret object (an empty string is treated as an `Opaque` type).

Kubernetes doesn't impose any constraints on the type name. However, if you are using one of the built-in types, you must meet all the requirements defined for that type.

If you are defining a type of Secret that's for public use, follow the convention and structure the Secret type to have your domain name before the name, separated by a `/`. For example: `cloud-hosting.example.net/cloud-api-credentials`.

Opaque Secrets

`Opaque` is the default Secret type if you don't explicitly specify a type in a Secret manifest. When you create a Secret using `kubectl`, you must use the `generic` subcommand to indicate an `Opaque` Secret type. For example, the following command creates an empty Secret of type `Opaque`:

```
kubectl create secret generic empty-secret
kubectl get secret empty-secret
```

The output looks like:

```
NAME           TYPE     DATA   AGE
empty-secret   Opaque   0      2m6s
```

The `DATA` column shows the number of data items stored in the Secret. In this case, `0` means you have created an empty Secret.

ServiceAccount token Secrets

A `kubernetes.io/service-account-token` type of Secret is used to store a token credential that identifies a ServiceAccount. This is a legacy mechanism that provides long-lived ServiceAccount credentials to Pods.

In Kubernetes v1.22 and later, the recommended approach is to obtain a short-lived, automatically rotating ServiceAccount token by using the `TokenRequest` API instead. You can get these short-lived tokens using the following methods:

- Call the `TokenRequest` API either directly or by using an API client like `kubectl`. For example, you can use the `kubectl create token` command.
- Request a mounted token in a projected volume in your Pod manifest. Kubernetes creates the token and mounts it in the Pod. The token is automatically invalidated when the Pod that it's mounted in is deleted. For details, see Launch a Pod using service account token projection.

> **Note:**
> You should only create a ServiceAccount token Secret if you can't use the `TokenRequest` API to obtain a token, and the security exposure of persisting a non-expiring token credential in a readable API object is acceptable to you. For instructions, see Manually create a long-lived API token for a ServiceAccount.

When using this Secret type, you need to ensure that the `kubernetes.io/service-account.name` annotation is set to an existing ServiceAccount name. If you are creating both the ServiceAccount and the Secret objects, you should create the ServiceAccount object first.

After the Secret is created, a Kubernetes controller fills in some other fields such as the `kubernetes.io/service-account.uid` annotation, and the `token` key in the `data` field, which is populated with an authentication token.

The following example configuration declares a ServiceAccount token Secret:

secret/serviceaccount-token-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-sa-sample
  annotations:
    kubernetes.io/service-account.name: "sa-name"
type: kubernetes.io/service-account-token
data:
  extra: YmFyCg==
```

After creating the Secret, wait for Kubernetes to populate the `token` key in the `data` field.

See the ServiceAccount documentation for more information on how ServiceAccounts work. You can also check the `automountServiceAccountToken` field and the `serviceAccountName` field of the `Pod` for information on referencing ServiceAccount credentials from within Pods.

Docker config Secrets

If you are creating a Secret to store credentials for accessing a container image registry, you must use one of the following `type` values for that Secret:

- `kubernetes.io/dockercfg` : store a serialized `~/.dockercfg` which is the legacy format for configuring Docker command line. The Secret `data` field contains a `.dockercfg` key whose value is the content of a base64 encoded `~/.dockercfg` file.
- `kubernetes.io/dockerconfigjson` : store a serialized JSON that follows the same format rules as the `~/.docker/config.json` file, which is a new format for `~/.dockercfg` . The Secret `data` field must contain a `.dockerconfigjson` key for which the value is the content of a base64 encoded `~/.docker/config.json` file.

Below is an example for a `kubernetes.io/dockercfg` type of Secret:

secret/dockercfg-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-dockercfg
type: kubernetes.io/dockercfg
data:
  .dockercfg: |
    eyJhdXRocyI6eyJodHRwczovL2V4YW1wbGUvdjEvIjp7ImF1dGgiOiJvcGVuc2VzYW1lIn19fQo=
```

> **Note:**
> If you do not want to perform the base64 encoding, you can choose to use the `stringData` field instead.

When you create Docker config Secrets using a manifest, the API server checks whether the expected key exists in the `data` field, and it verifies if the value provided can be

parsed as a valid JSON. The API server doesn't validate if the JSON actually is a Docker config file.

You can also use `kubectl` to create a Secret for accessing a container registry, such as when you don't have a Docker configuration file:

```
kubectl create secret docker-registry secret-tiger-docker \
  --docker-email=tiger@acme.example \
  --docker-username=tiger \
  --docker-password=pass1234 \
  --docker-server=my-registry.example:5000
```

This command creates a Secret of type `kubernetes.io/dockerconfigjson`.

Retrieve the `.data.dockerconfigjson` field from that new Secret and decode the data:

```
kubectl get secret secret-tiger-docker -o jsonpath='{.data.*}' | base64 -d
```

The output is equivalent to the following JSON document (which is also a valid Docker configuration file):

```
{
  "auths": {
    "my-registry.example:5000": {
      "username": "tiger",
      "password": "pass1234",
      "email": "tiger@acme.example",
      "auth": "dGlnZXI6cGFzczEyMzQ="
    }
  }
}
```

> **Caution:**
>
> The `auth` value there is base64 encoded; it is obscured but not secret. Anyone who can read that Secret can learn the registry access bearer token.

> It is suggested to use credential providers to dynamically and securely provide pull secrets on-demand.

Basic authentication Secret

The `kubernetes.io/basic-auth` type is provided for storing credentials needed for basic authentication. When using this Secret type, the `data` field of the Secret must contain one of the following two keys:

- `username` : the user name for authentication
- `password` : the password or token for authentication

Both values for the above two keys are base64 encoded strings. You can alternatively provide the clear text content using the `stringData` field in the Secret manifest.

The following manifest is an example of a basic authentication Secret:

secret/basicauth-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-basic-auth
type: kubernetes.io/basic-auth
stringData:
  username: admin # required field for kubernetes.io/basic-auth
  password: t0p-Secret # required field for kubernetes.io/basic-auth
```

> **Note:**
> The `stringData` field for a Secret does not work well with server-side apply.

The basic authentication Secret type is provided only for convenience. You can create an `Opaque` type for credentials used for basic authentication. However, using the defined and public Secret type (`kubernetes.io/basic-auth`) helps other people to understand the purpose of your Secret, and sets a convention for what key names to expect.

SSH authentication Secrets

The builtin type `kubernetes.io/ssh-auth` is provided for storing data used in SSH authentication. When using this Secret type, you will have to specify a `ssh-privatekey` key-value pair in the `data` (or `stringData`) field as the SSH credential to use.

The following manifest is an example of a Secret used for SSH public/private key authentication:

secret/ssh-auth-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-ssh-auth
type: kubernetes.io/ssh-auth
data:
  # the data is abbreviated in this example
  ssh-privatekey: |
    UG91cmluZzYlRW1vdGljb24lU2N1YmE=
```

The SSH authentication Secret type is provided only for convenience. You can create an `Opaque` type for credentials used for SSH authentication. However, using the defined and public Secret type (`kubernetes.io/ssh-auth`) helps other people to understand the purpose of your Secret, and sets a convention for what key names to expect. The Kubernetes API verifies that the required keys are set for a Secret of this type.

> **Caution:**
>
> SSH private keys do not establish trusted communication between an SSH client and host server on their own. A secondary means of establishing trust is needed to mitigate "man in the middle" attacks, such as a `known_hosts` file added to a ConfigMap.

TLS Secrets

The `kubernetes.io/tls` Secret type is for storing a certificate and its associated key that are typically used for TLS.

One common use for TLS Secrets is to configure encryption in transit for an Ingress, but you can also use it with other resources or directly in your workload. When using this type of Secret, the `tls.key` and the `tls.crt` key must be provided in the `data` (or `stringData`)

field of the Secret configuration, although the API server doesn't actually validate the values for each key.

As an alternative to using `stringData`, you can use the `data` field to provide the base64 encoded certificate and private key. For details, see Constraints on Secret names and data.

The following YAML contains an example config for a TLS Secret:

secret/tls-auth-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-tls
type: kubernetes.io/tls
data:
  # values are base64 encoded, which obscures them but does NOT provide
  # any useful level of confidentiality
  # Replace the following values with your own base64-encoded certificate and key.
  tls.crt: "REPLACE_WITH_BASE64_CERT"
  tls.key: "REPLACE_WITH_BASE64_KEY"
```

The TLS Secret type is provided only for convenience. You can create an `Opaque` type for credentials used for TLS authentication. However, using the defined and public Secret type (`kubernetes.io/tls`) helps ensure the consistency of Secret format in your project. The API server verifies if the required keys are set for a Secret of this type.

To create a TLS Secret using `kubectl`, use the `tls` subcommand:

```
kubectl create secret tls my-tls-secret \
  --cert=path/to/cert/file \
  --key=path/to/key/file
```

The public/private key pair must exist before hand. The public key certificate for `--cert` must be .PEM encoded and must match the given private key for `--key`.

Bootstrap token Secrets

The `bootstrap.kubernetes.io/token` Secret type is for tokens used during the node bootstrap process. It stores tokens used to sign well-known ConfigMaps.

A bootstrap token Secret is usually created in the `kube-system` namespace and named in the form `bootstrap-token-<token-id>` where `<token-id>` is a 6 character string of the token ID.

As a Kubernetes manifest, a bootstrap token Secret might look like the following:

secret/bootstrap-token-secret-base64.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: bootstrap-token-5emitj
  namespace: kube-system
type: bootstrap.kubernetes.io/token
data:
  auth-extra-groups: c3lzdGVtOmJvb3RzdHJhcHBlcnM6a3ViZWFkbTpkZWZhdWx0LW5vZGUtdG9rZW4=
  expiration: MjAyMC0wOS0xM1QwNDozOToxMFo=
  token-id: NWVtaXRq
  token-secret: a3E0Z2lodnN6emduMXAwcg==
  usage-bootstrap-authentication: dHJ1ZQ==
  usage-bootstrap-signing: dHJ1ZQ==
```

A bootstrap token Secret has the following keys specified under `data`:

- `token-id`: A random 6 character string as the token identifier. Required.
- `token-secret`: A random 16 character string as the actual token Secret. Required.
- `description`: A human-readable string that describes what the token is used for. Optional.
- `expiration`: An absolute UTC time using RFC3339 specifying when the token should be expired. Optional.
- `usage-bootstrap-<usage>`: A boolean flag indicating additional usage for the bootstrap token.
- `auth-extra-groups`: A comma-separated list of group names that will be authenticated as in addition to the `system:bootstrappers` group.

You can alternatively provide the values in the `stringData` field of the Secret without base64 encoding them:

secret/bootstrap-token-secret-literal.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  # Note how the Secret is named
  name: bootstrap-token-5emitj
  # A bootstrap token Secret usually resides in the kube-system namespace
  namespace: kube-system
type: bootstrap.kubernetes.io/token
stringData:
  auth-extra-groups: "system:bootstrappers:kubeadm:default-node-token"
  expiration: "2020-09-13T04:39:10Z"
  # This token ID is used in the name
  token-id: "5emitj"
  token-secret: "kq4gihvszzgn1p0r"
  # This token can be used for authentication
  usage-bootstrap-authentication: "true"
  # and it can be used for signing
  usage-bootstrap-signing: "true"
```

> **Note:**
> The `stringData` field for a Secret does not work well with server-side apply.

Working with Secrets

Creating a Secret

There are several options to create a Secret:

- Use `kubectl`
- Use a configuration file
- Use the Kustomize tool

Constraints on Secret names and data

The name of a Secret object must be a valid DNS subdomain name.

You can specify the `data` and/or the `stringData` field when creating a configuration file for a Secret. The `data` and the `stringData` fields are optional. The values for all keys in the `data` field have to be base64-encoded strings. If the conversion to base64 string is not desirable, you can choose to specify the `stringData` field instead, which accepts arbitrary strings as values.

The keys of `data` and `stringData` must consist of alphanumeric characters, `-`, `_` or `.`. All key-value pairs in the `stringData` field are internally merged into the `data` field. If a key appears in both the `data` and the `stringData` field, the value specified in the `stringData` field takes precedence.

Size limit

Individual Secrets are limited to 1MiB in size. This is to discourage creation of very large Secrets that could exhaust the API server and kubelet memory. However, creation of many smaller Secrets could also exhaust memory. You can use a resource quota to limit the number of Secrets (or other resources) in a namespace.

Editing a Secret

You can edit an existing Secret unless it is immutable. To edit a Secret, use one of the following methods:

- Use `kubectl`
- Use a configuration file

You can also edit the data in a Secret using the Kustomize tool. However, this method creates a new `Secret` object with the edited data.

Depending on how you created the Secret, as well as how the Secret is used in your Pods, updates to existing `Secret` objects are propagated automatically to Pods that use the data. For more information, refer to Using Secrets as files from a Pod section.

Using a Secret

Secrets can be mounted as data volumes or exposed as environment variables to be used by a container in a Pod. Secrets can also be used by other parts of the system, without being directly exposed to the Pod. For example, Secrets can hold credentials that other parts of the system should use to interact with external systems on your behalf.

Secret volume sources are validated to ensure that the specified object reference actually points to an object of type Secret. Therefore, a Secret needs to be created before any

Pods that depend on it.

If the Secret cannot be fetched (perhaps because it does not exist, or due to a temporary lack of connection to the API server) the kubelet periodically retries running that Pod. The kubelet also reports an Event for that Pod, including details of the problem fetching the Secret.

Optional Secrets

When you reference a Secret in a Pod, you can mark the Secret as *optional*, such as in the following example. If an optional Secret doesn't exist, Kubernetes ignores it.

secret/optional-secret.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    secret:
      secretName: mysecret
      optional: true
```

By default, Secrets are required. None of a Pod's containers will start until all non-optional Secrets are available.

If a Pod references a specific key in a non-optional Secret and that Secret does exist, but is missing the named key, the Pod fails during startup.

Using Secrets as files from a Pod

If you want to access data from a Secret in a Pod, one way to do that is to have Kubernetes make the value of that Secret be available as a file inside the filesystem of one or more of the Pod's containers.

For instructions, refer to Create a Pod that has access to the secret data through a Volume.

When a volume contains data from a Secret, and that Secret is updated, Kubernetes tracks this and updates the data in the volume, using an eventually-consistent approach.

> **Note:**
> A container using a Secret as a subPath volume mount does not receive automated Secret updates.

The kubelet keeps a cache of the current keys and values for the Secrets that are used in volumes for pods on that node. You can configure the way that the kubelet detects changes from the cached values. The `configMapAndSecretChangeDetectionStrategy` field in the kubelet configuration controls which strategy the kubelet uses. The default strategy is `Watch`.

Updates to Secrets can be either propagated by an API watch mechanism (the default), based on a cache with a defined time-to-live, or polled from the cluster API server on each kubelet synchronisation loop.

As a result, the total delay from the moment when the Secret is updated to the moment when new keys are projected to the Pod can be as long as the kubelet sync period + cache propagation delay, where the cache propagation delay depends on the chosen cache type (following the same order listed in the previous paragraph, these are: watch propagation delay, the configured cache TTL, or zero for direct polling).

Using Secrets as environment variables

To use a Secret in an environment variable in a Pod:

1. For each container in your Pod specification, add an environment variable for each Secret key that you want to use to the `env[].valueFrom.secretKeyRef` field.
2. Modify your image and/or command line so that the program looks for values in the specified environment variables.

For instructions, refer to Define container environment variables using Secret data.

It's important to note that the range of characters allowed for environment variable names in pods is restricted. If any keys do not meet the rules, those keys are not made available to your container, though the Pod is allowed to start.

Container image pull Secrets

If you want to fetch container images from a private repository, you need a way for the kubelet on each node to authenticate to that repository. You can configure *image pull Secrets* to make this possible. These Secrets are configured at the Pod level.

Using imagePullSecrets

The `imagePullSecrets` field is a list of references to Secrets in the same namespace. You can use an `imagePullSecrets` to pass a Secret that contains a Docker (or other) image registry password to the kubelet. The kubelet uses this information to pull a private image on behalf of your Pod. See the PodSpec API for more information about the `imagePullSecrets` field.

Manually specifying an imagePullSecret

You can learn how to specify `imagePullSecrets` from the container images documentation.

Arranging for imagePullSecrets to be automatically attached

You can manually create `imagePullSecrets`, and reference these from a ServiceAccount. Any Pods created with that ServiceAccount or created with that ServiceAccount by default, will get their `imagePullSecrets` field set to that of the service account. See Add ImagePullSecrets to a service account for a detailed explanation of that process.

Using Secrets with static Pods

You cannot use ConfigMaps or Secrets with static Pods.

Immutable Secrets

ⓘ **FEATURE STATE:** Kubernetes v1.21 [stable]

Kubernetes lets you mark specific Secrets (and ConfigMaps) as *immutable*. Preventing changes to the data of an existing Secret has the following benefits:

- protects you from accidental (or unwanted) updates that could cause applications outages
- (for clusters that extensively use Secrets - at least tens of thousands of unique Secret to Pod mounts), switching to immutable Secrets improves the performance of your cluster by significantly reducing load on kube-apiserver. The kubelet does not need to maintain a [watch] on any Secrets that are marked as immutable.

Marking a Secret as immutable

You can create an immutable Secret by setting the `immutable` field to `true`. For example,

```yaml
apiVersion: v1
kind: Secret
metadata: ...
data: ...
immutable: true
```

You can also update any existing mutable Secret to make it immutable.

> **Note:**
> Once a Secret or ConfigMap is marked as immutable, it is *not* possible to revert this change nor to mutate the contents of the `data` field. You can only delete and recreate the Secret. Existing Pods maintain a mount point to the deleted Secret - it is recommended to recreate these pods.

Information security for Secrets

Although ConfigMap and Secret work similarly, Kubernetes applies some additional protection for Secret objects.

Secrets often hold values that span a spectrum of importance, many of which can cause escalations within Kubernetes (e.g. service account tokens) and to external systems. Even

if an individual app can reason about the power of the Secrets it expects to interact with, other apps within the same namespace can render those assumptions invalid.

Authorization configuration affects how Secret data can be accessed within a namespace. For example, granting **list** or **watch** permissions on Secrets allows a subject to read all Secret data in that namespace, not only the Secrets explicitly referenced by its Pods. Restrict access to the minimum set of permissions required for a workload to function, and avoid granting broad roles such as `cluster-admin` unless required for administrative purposes.

Also see the Authorization documentation.

A Secret is only sent to a node if a Pod on that node requires it. For mounting Secrets into Pods, the kubelet stores a copy of the data into a `tmpfs` so that the confidential data is not written to durable storage. Once the Pod that depends on the Secret is deleted, the kubelet deletes its local copy of the confidential data from the Secret.

There may be several containers in a Pod. By default, containers you define only have access to the default ServiceAccount and its related Secret. You must explicitly define environment variables or map a volume into a container in order to provide access to any other Secret.

There may be Secrets for several Pods on the same node. However, only the Secrets that a Pod requests are potentially visible within its containers. Therefore, one Pod does not have access to the Secrets of another Pod.

Configure least-privilege access to Secrets

To enhance the security measures around Secrets, use separate namespaces to isolate access to mounted secrets.

> **Warning:**
>
> Any containers that run with `privileged: true` on a node can access all Secrets used on that node.

What's next

- For guidelines to manage and improve the security of your Secrets, refer to Good practices for Kubernetes Secrets.

- Learn how to manage Secrets using `kubectl`
- Learn how to manage Secrets using config file
- Learn how to manage Secrets using kustomize
- Read the API reference for `Secret`

3 - Resource Management for Pods and Containers

When you specify a Pod, you can optionally specify how much of each resource a container needs. The most common resources to specify are CPU and memory (RAM); there are others.

When you specify the resource *request* for containers in a Pod, the kube-scheduler uses this information to decide which node to place the Pod on. When you specify a resource *limit* for a container, the kubelet enforces those limits so that the running container is not allowed to use more of that resource than the limit you set. The kubelet also reserves at least the *request* amount of that system resource specifically for that container to use.

Requests and limits

If the node where a Pod is running has enough of a resource available, it's possible (and allowed) for a container to use more resource than its `request` for that resource specifies.

For example, if you set a `memory` request of 256 MiB for a container, and that container is in a Pod scheduled to a Node with 8GiB of memory and no other Pods, then the container can try to use more RAM.

Limits are a different story. Both `cpu` and `memory` limits are applied by the kubelet (and container runtime), and are ultimately enforced by the kernel. On Linux nodes, the Linux kernel enforces limits with cgroups. The behavior of `cpu` and `memory` limit enforcement is slightly different.

`cpu` limits are enforced by CPU throttling. When a container approaches its `cpu` limit, the kernel will restrict access to the CPU corresponding to the container's limit. Thus, a `cpu` limit is a hard limit the kernel enforces. Containers may not use more CPU than is specified in their `cpu` limit.

`memory` limits are enforced by the kernel with out of memory (OOM) kills. When a container uses more than its `memory` limit, the kernel may terminate it. However, terminations only happen when the kernel detects memory pressure. Thus, a container that over allocates memory may not be immediately killed. This means `memory` limits are enforced reactively. A container may use more memory than its `memory` limit, but if it does, it may get killed.

> **Note:**
> There is an alpha feature `MemoryQoS` which adds memory throttling and optional tiered memory reservation on Linux nodes using cgroup v2. For details, see Memory QoS with cgroup v2.

> **Note:**
> If you specify a limit for a resource, but do not specify any request, and no admission-time mechanism has applied a default request for that resource, then Kubernetes copies the limit you specified and uses it as the requested value for the resource.

Resource types

A *resource type* has a base unit and can be requested, limited, or both. Kubernetes has the following built-in resource types:

Resource type	Description	Base unit
`cpu`	Compute processing	cpu (core)
`memory`	RAM	Bytes
`ephemeral-storage`	Local ephemeral storage	Bytes
`hugepages-<size>`	Huge pages (Linux only)	Bytes

Clusters can also provide extended resources (resources with a custom name, typically exposed by device plugins).

Huge pages

For Linux workloads, you can specify *huge page* resources. Huge pages are a Linux-specific feature where the node kernel allocates blocks of memory that are much larger than the default page size.

For example, on a system where the default page size is 4KiB, you could specify a limit, `hugepages-2Mi: 80Mi` . If the container tries allocating over 40 2MiB huge pages (a total of 80 MiB), that allocation fails.

> **Note:**
> You cannot overcommit `hugepages-*` resources. This is different from the `memory` and `cpu` resources.

CPU and memory are collectively referred to as *compute resources*, or *resources*. Compute resources are measurable quantities that can be requested, allocated, and consumed. They are distinct from API resources. API resources, such as Pods and Services are objects that can be read and modified through the Kubernetes API server.

Resource requests and limits of Pod and container

For each container, you can specify resource limits and requests, including the following:

- `spec.containers[].resources.limits.cpu`
- `spec.containers[].resources.limits.memory`
- `spec.containers[].resources.limits.ephemeral-storage`
- `spec.containers[].resources.limits.hugepages-<size>`
- `spec.containers[].resources.requests.cpu`
- `spec.containers[].resources.requests.memory`
- `spec.containers[].resources.requests.ephemeral-storage`
- `spec.containers[].resources.requests.hugepages-<size>`

Although you can only specify requests and limits for individual containers, it is also useful to think about the overall resource requests and limits for a Pod. For a particular resource, a *Pod resource request/limit* is the sum of the resource requests/limits of that type for each container in the Pod.

Pod-level resource specification

ⓘ **FEATURE STATE:** Kubernetes v1.34 [beta](enabled by default)

Provided your cluster has the `PodLevelResources` feature gate enabled, you can specify resource requests and limits at the Pod level. At the Pod level, Kubernetes 1.36 only supports resource requests or limits for specific resource types: `cpu` and / or `memory` and / or `hugepages`. With this feature, Kubernetes allows you to declare an overall resource budget for the Pod, which is especially helpful when dealing with a large number of containers where it can be difficult to accurately gauge individual resource needs. Additionally, it enables containers within a Pod to share idle resources with each other, improving resource utilization.

For a Pod, you can specify resource limits and requests for CPU and memory by including the following:

- `spec.resources.limits.cpu`
- `spec.resources.limits.memory`
- `spec.resources.limits.hugepages-<size>`
- `spec.resources.requests.cpu`
- `spec.resources.requests.memory`
- `spec.resources.requests.hugepages-<size>`

Resource units in Kubernetes

CPU resource units

Limits and requests for CPU resources are measured in *cpu* units. In Kubernetes, 1 CPU unit is equivalent to **1 physical CPU core**, or **1 virtual core**, depending on whether the node is a physical host or a virtual machine running inside a physical machine.

Fractional requests are allowed. When you define a container with `spec.containers[].resources.requests.cpu` set to `0.5`, you are requesting half as much CPU time compared to if you asked for `1.0` CPU. For CPU resource units, the quantity expression `0.1` is equivalent to the expression `100m`, which can be read as "one hundred millicpu". Some people say "one hundred millicores", and this is understood to mean the same thing.

CPU resource is always specified as an absolute amount of resource, never as a relative amount. For example, `500m` CPU represents the roughly same amount of computing power whether that container runs on a single-core, dual-core, or 48-core machine.

> **Note:**

> Kubernetes doesn't allow you to specify CPU resources with a precision finer than `1m` or `0.001` CPU. To avoid accidentally using an invalid CPU quantity, it's useful to specify CPU units using the milliCPU form instead of the decimal form when using less than 1 CPU unit.
>
> For example, you have a Pod that uses `5m` or `0.005` CPU and would like to decrease its CPU resources. By using the decimal form, it's harder to spot that `0.0005` CPU is an invalid value, while by using the milliCPU form, it's easier to spot that `0.5m` is an invalid value.

Memory resource units

Limits and requests for `memory` are measured in bytes. You can express memory as a plain integer or as a fixed-point number using one of these quantity suffixes: E, P, T, G, M, k. You can also use the power-of-two equivalents: Ei, Pi, Ti, Gi, Mi, Ki. The Kubernetes API also allows m as a suffix (for millibytes: 1/1000 of a byte), but this isn't useful to specify: you must always assign whole numbers of bytes, or sometimes larger chunks such as multiples of 1 gibibyte.

Here are some examples of memory quantities that represent roughly the same value:

```
128974848, 129e6, 129M,  128974848000m, 123Mi
```

Pay attention to the case of the suffixes. "M" means megabytes, while "m" means millibytes. If you request `400m` of memory, this is a request for 0.4 bytes. Someone who types that probably meant to ask for 400 mebibytes (`400Mi`) or 400 megabytes (`400M`).

Container resources example

The following Pod has two containers. Both containers are defined with a request for 0.25 CPU and 64MiB (2^{26} bytes) of memory. Each container has a limit of 0.5 CPU and 128MiB of memory. You can say the Pod has a request of 0.5 CPU and 128 MiB of memory, and a limit of 1 CPU and 256MiB of memory.

```
---
apiVersion: v1
```

```yaml
kind: Pod
metadata:
  name: frontend
spec:
  containers:
  - name: app
    image: images.my-company.example/app:v4
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
  - name: log-aggregator
    image: images.my-company.example/log-aggregator:v6
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Pod resources example

ⓘ **FEATURE STATE:** Kubernetes v1.34 [beta](enabled by default)

This feature can be enabled by setting the `PodLevelResources` feature gate. The following Pod has an explicit request of 1 CPU and 100 MiB of memory, and an explicit limit of 1 CPU and 200 MiB of memory. The `pod-resources-demo-ctr-1` container has explicit requests and limits set. However, the `pod-resources-demo-ctr-2` container will simply share the resources available within the Pod resource boundaries, as it does not have explicit requests and limits set.

`pods/resource/pod-level-resources.yaml`

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-resources-demo
  namespace: pod-resources-example
spec:
  resources:
    limits:
      cpu: "1"
      memory: "200Mi"
    requests:
      cpu: "1"
      memory: "100Mi"
  containers:
  - name: pod-resources-demo-ctr-1
    image: nginx
    resources:
      limits:
        cpu: "0.5"
        memory: "100Mi"
      requests:
        cpu: "0.5"
        memory: "50Mi"
  - name: pod-resources-demo-ctr-2
    image: fedora
    command:
    - sleep
    - inf
```

How Pods with resource requests are scheduled

When you create a Pod, the Kubernetes scheduler selects a node for the Pod to run on. Each node has a maximum capacity for each of the resource types: the amount of CPU and memory it can provide for Pods. The scheduler ensures that, for each resource type, the sum of the resource requests of the scheduled containers is less than the capacity of the node. Note that although actual memory or CPU resource usage on nodes is very low, the scheduler still refuses to place a Pod on a node if the capacity check fails. This protects against a resource shortage on a node when resource usage later increases, for example, during a daily peak in request rate.

How Kubernetes applies resource requests and limits

When the kubelet starts a container as part of a Pod, the kubelet passes that container's requests and limits for memory and CPU to the container runtime.

On Linux, the container runtime typically configures kernel cgroups that apply and enforce the limits you defined.

- The CPU limit defines a hard ceiling on how much CPU time the container can use. During each scheduling interval (time slice), the Linux kernel checks to see if this limit is exceeded; if so, the kernel waits before allowing that cgroup to resume execution.
- The CPU request typically defines a weighting. If several different containers (cgroups) want to run on a contended system, workloads with larger CPU requests are allocated more CPU time than workloads with small requests.
- The memory request is mainly used during (Kubernetes) Pod scheduling. On a node that uses cgroups v2, the container runtime might use the memory request as a hint to set `memory.min` and `memory.low`.
- The memory limit defines a memory limit for that cgroup. If the container tries to allocate more memory than this limit, the Linux kernel out-of-memory subsystem activates and, typically, intervenes by stopping one of the processes in the container that tried to allocate memory. If that process is the container's PID 1, and the container is marked as restartable, Kubernetes restarts the container.
- The memory limit for the Pod or container can also apply to pages in memory backed volumes, such as an `emptyDir`. The kubelet tracks `tmpfs` emptyDir volumes as container memory use, rather than as local ephemeral storage. When using memory backed `emptyDir`, be sure to check the notes below.

If a container exceeds its memory request and the node that it runs on becomes short of memory overall, it is likely that the Pod the container belongs to will be evicted.

A container might or might not be allowed to exceed its CPU limit for extended periods of time. However, container runtimes don't terminate Pods or containers for excessive CPU usage.

To determine whether a container cannot be scheduled or is being killed due to resource limits, see the Troubleshooting section.

Resizing container resources

After creating a Pod, you may need to adjust its CPU or memory resources based on actual usage patterns. Kubernetes provides two approaches for resizing Pod resources:

In-place resize

ⓘ **FEATURE STATE:** Kubernetes v1.35 [stable](enabled by default)

You can modify the CPU and memory `requests` and `limits` of containers in a running Pod without recreating it. This is called *in-place Pod vertical scaling* or *in-place Pod resize*. To perform an in-place resize, update the container's resource specifications using the Pod's `/resize` subresource. You can control whether a container restart is required by setting the `resizePolicy` field in the container specification.

> **Note:**
> In-place resize currently applies to container-level resources. For resizing Pod-level resources, see Resize Pod CPU and Memory Resources.

Resizing by launching replacement Pods

The cloud native approach to changing a Pod's resources is to update the Pod template in the workload object (such as a Deployment or StatefulSet) and let the workload's controller replace Pods with new ones that have the updated resources. This approach works with any Kubernetes version and can change any Pod specification.

For more details about Pod resizing, see Resizing Pods. For detailed instructions on in-place resize, see Resize CPU and Memory Resources assigned to Containers. You can also use the Vertical Pod Autoscaler to automatically manage Pod resource recommendations.

Monitoring compute & memory resource usage

The kubelet reports the resource usage of a Pod as part of the Pod `status`.

If optional tools for monitoring are available in your cluster, then Pod resource usage can be retrieved either from the Metrics API directly or from your monitoring tools.

Considerations for memory backed `emptyDir` volumes

> **Caution:**
>
> If you do not specify a `sizeLimit` for an `emptyDir` volume, that volume may consume up to that pod's memory limit (`Pod.spec.containers[].resources.limits.memory`). If you do not set a memory limit, the pod has no upper bound on memory consumption, and can consume all available memory on the node. Kubernetes schedules pods based on resource requests (`Pod.spec.containers[].resources.requests`) and will not consider memory usage above the request when deciding if another pod can fit on a given node. This can result in a denial of service and cause the OS to do out-of-memory (OOM) handling. It is possible to create any number of `emptyDir`s that could potentially consume all available memory on the node, making OOM more likely.

From the perspective of memory management, there are some similarities between when a process uses memory as a work area and when using memory-backed `emptyDir`. But when using memory as a volume, like memory-backed `emptyDir`, there are additional points below that you should be careful of:

- Files stored on a memory-backed volume are almost entirely managed by the user application. Unlike when used as a work area for a process, you can not rely on things like language-level garbage collection.
- The purpose of writing files to a volume is to save data or pass it between applications. Neither Kubernetes nor the OS may automatically delete files from a volume, so memory used by those files can not be reclaimed when the system or the pod are under memory pressure.
- A memory-backed `emptyDir` is useful because of its performance, but memory is generally much smaller in size and much higher in cost than other storage media, such as disks or SSDs. Using large amounts of memory for `emptyDir` volumes may affect the normal operation of your pod or of the whole node, so should be used carefully.

If you are administering a cluster or namespace, you can also set ResourceQuota that limits memory use; you may also want to define a LimitRange for additional enforcement. If you specify a `spec.containers[].resources.limits.memory` for each Pod, then the maximum size of an `emptyDir` volume will be the pod's memory limit.

As an alternative, a cluster administrator can enforce size limits for `emptyDir` volumes in new Pods using a policy mechanism such as ValidatingAdmissionPolicy.

Local ephemeral storage

For general concepts about local ephemeral storage and hints about configuring the requests and/or limits of ephemeral storage for a container, please check the local ephemeral storage page.

Resource monitoring for local ephemeral storage

The kubelet can measure how much local ephemeral storage is being used. It does this as long as you have enabled local ephemeral storage capacity isolation.

Kubernetes tracks the amount of ephemeral storage a Pod uses from the following:

- Writing to the container's writable layer (rootfs), container images, or both.
- Writing to local `emptyDir` volumes.
- The Pod's own logs (usually stored under `/var/log/pods`).
- System files managed by Kubernetes that are mapped into the Pod, such as `/etc/hosts`.

Extended resources

Extended resources are fully-qualified resource names outside the `kubernetes.io` domain. They allow cluster operators to advertise and users to consume the non-Kubernetes-built-in resources.

There are two steps required to use Extended Resources. First, the cluster operator must advertise an Extended Resource. Second, users must request the Extended Resource in Pods.

Managing extended resources

Node-level extended resources

Node-level extended resources are tied to nodes.

Device plugin managed resources

See Device Plugin for how to advertise device plugin managed resources on each node.

Other resources

To advertise a new node-level extended resource, the cluster operator can submit a `PATCH` HTTP request to the API server to specify the available quantity in the `status.capacity` for a node in the cluster. After this operation, the node's `status.capacity` will include a new resource. The `status.allocatable` field is updated automatically with the new resource asynchronously by the kubelet.

Because the scheduler uses the node's `status.allocatable` value when evaluating Pod fitness, the scheduler only takes account of the new value after that asynchronous update. There may be a short delay between patching the node capacity with a new resource and the time when the first Pod that requests the resource can be scheduled on that node.

Example:

Here is an example showing how to use `curl` to form an HTTP request that advertises five "example.com/foo" resources on node `k8s-node-1` whose master is `k8s-master`.

```
curl --header "Content-Type: application/json-patch+json" \
--request PATCH \
--data '[{"op": "add", "path": "/status/capacity/example.com~1foo", "value": "5"}]' \
http://k8s-master:8080/api/v1/nodes/k8s-node-1/status
```

> **Note:**
> In the preceding request, `~1` is the encoding for the character `/` in the patch path. The operation path value in JSON-Patch is interpreted as a JSON-Pointer. For more details, see IETF RFC 6901, section 3.

Cluster-level extended resources

Cluster-level extended resources are not tied to nodes. They are usually managed by scheduler extenders, which handle the resource consumption and resource quota.

You can specify the extended resources that are handled by scheduler extenders in scheduler configuration

Example:

The following configuration for a scheduler policy indicates that the cluster-level extended resource "example.com/foo" is handled by the scheduler extender.

- The scheduler sends a Pod to the scheduler extender only if the Pod requests "example.com/foo".
- The `ignoredByScheduler` field specifies that the scheduler does not check the "example.com/foo" resource in its `PodFitsResources` predicate.

```
{
  "kind": "Policy",
  "apiVersion": "v1",
  "extenders": [
    {
      "urlPrefix":"<extender-endpoint>",
      "bindVerb": "bind",
      "managedResources": [
        {
          "name": "example.com/foo",
          "ignoredByScheduler": true
        }
      ]
    }
  ]
}
```

Extended resources allocation by DRA

Extended resources allocation by DRA allows cluster administrators to specify an `extendedResourceName` in DeviceClass, then the devices matching the DeviceClass can be requested from a pod's extended resource requests. Read more about Extended Resource allocation by DRA.

Consuming extended resources

Users can consume extended resources in Pod specs like CPU and memory. The scheduler takes care of the resource accounting so that no more than the available amount is simultaneously allocated to Pods.

The API server restricts quantities of extended resources to whole numbers. Examples of *valid* quantities are `3`, `3000m` and `3Ki`. Examples of *invalid* quantities are `0.5` and `1500m` (because `1500m` would result in `1.5`).

> **Note:**

> Extended resources replace Opaque Integer Resources. Users can use any domain name prefix other than `kubernetes.io` which is reserved.

To consume an extended resource in a Pod, include the resource name as a key in the `spec.containers[].resources.limits` map in the container spec.

> **Note:**
> Extended resources cannot be overcommitted, so request and limit must be equal if both are present in a container spec.

A Pod is scheduled only if all of the resource requests are satisfied, including CPU, memory and any extended resources. The Pod remains in the `PENDING` state as long as the resource request cannot be satisfied.

Example:

The Pod below requests 2 CPUs and 1 "example.com/foo" (an extended resource).

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: my-container
    image: myimage
    resources:
      requests:
        cpu: 2
        example.com/foo: 1
      limits:
        example.com/foo: 1
```

PID limiting

Process ID (PID) limits allow for the configuration of a kubelet to limit the number of PIDs that a given Pod can consume. See PID Limiting for information.

Troubleshooting

My Pods are pending with event message `FailedScheduling`

If the scheduler cannot find any node where a Pod can fit, the Pod remains unscheduled until a place can be found. An Event is produced each time the scheduler fails to find a place for the Pod. You can use `kubectl` to view the events for a Pod; for example:

```
kubectl describe pod frontend | grep -A 9999999999 Events
```

```
Events:
  Type     Reason            Age   From               Message
  ----     ------            ----  ----               -------
  Warning  FailedScheduling  23s   default-scheduler  0/42 nodes available: insuffici
```

In the preceding example, the Pod named "frontend" fails to be scheduled due to insufficient CPU resource on any node. Similar error messages can also suggest failure due to insufficient memory (PodExceedsFreeMemory). In general, if a Pod is pending with a message of this type, there are several things to try:

- Add more nodes to the cluster.
- Terminate unneeded Pods to make room for pending Pods.
- Check that the Pod is not larger than all the nodes. For example, if all the nodes have a capacity of `cpu: 1`, then a Pod with a request of `cpu: 1.1` will never be scheduled.
- Check for node taints. If most of your nodes are tainted, and the new Pod does not tolerate that taint, the scheduler only considers placements onto the remaining nodes that don't have that taint.

You can check node capacities and amounts allocated with the `kubectl describe nodes` command. For example:

```
kubectl describe nodes e2e-test-node-pool-4lw4
```

```
Name:            e2e-test-node-pool-4lw4
[ ... lines removed for clarity ...]
Capacity:
 cpu:                           2
 memory:                        7679792Ki
 pods:                          110
Allocatable:
 cpu:                           1800m
 memory:                        7474992Ki
 pods:                          110
[ ... lines removed for clarity ...]
Non-terminated Pods:        (5 in total)
  Namespace    Name                                  CPU Requests  CPU Limits  Memory
  ---------    ----                                  ------------  ----------  ------
  kube-system  fluentd-gcp-v1.38-28bv1               100m (5%)     0 (0%)      200Mi
  kube-system  kube-dns-3297075139-61lj3             260m (13%)    0 (0%)      100Mi
  kube-system  kube-proxy-e2e-test-...               100m (5%)     0 (0%)      0 (0%)
  kube-system  monitoring-influxdb-grafana-v4-z1m12  200m (10%)    200m (10%)  600Mi
  kube-system  node-problem-detector-v0.1-fj7m3      20m (1%)      200m (10%)  20Mi (
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  CPU Requests    CPU Limits    Memory Requests    Memory Limits
  ------------    ----------    ---------------    -------------
  680m (34%)      400m (20%)    920Mi (11%)        1070Mi (13%)
```

In the preceding output, you can see that if a Pod requests more than 1.120 CPUs or more than 6.23Gi of memory, that Pod will not fit on the node.

By looking at the “Pods” section, you can see which Pods are taking up space on the node.

The amount of resources available to Pods is less than the node capacity because system daemons use a portion of the available resources. Within the Kubernetes API, each Node has a `.status.allocatable` field (see NodeStatus for details).

The `.status.allocatable` field describes the amount of resources that are available to Pods on that node (for example: 15 virtual CPUs and 7538 MiB of memory). For more information on node allocatable resources in Kubernetes, see Reserve Compute Resources for System Daemons.

You can configure resource quotas to limit the total amount of resources that a namespace can consume. Kubernetes enforces quotas for objects in particular namespace when there is a ResourceQuota in that namespace. For example, if you assign specific namespaces to different teams, you can add ResourceQuotas into those

namespaces. Setting resource quotas helps to prevent one team from using so much of any resource that this over-use affects other teams.

You should also consider what access you grant to that namespace: **full** write access to a namespace allows someone with that access to remove any resource, including a configured ResourceQuota.

My container is terminated

Your container might get terminated because it is resource-starved. To check whether a container is being killed because it is hitting a resource limit, call `kubectl describe pod` on the Pod of interest:

```
kubectl describe pod simmemleak-hra99
```

The output is similar to:

```
Name:                           simmemleak-hra99
Namespace:                      default
Image(s):                       saadali/simmemleak
Node:                           kubernetes-node-tf0f/10.240.216.66
Labels:                         name=simmemleak
Status:                         Running
Reason:
Message:
IP:                             10.244.2.75
Containers:
  simmemleak:
    Image:  saadali/simmemleak:latest
    Limits:
      cpu:          100m
      memory:       50Mi
    State:          Running
      Started:      Tue, 07 Jul 2019 12:54:41 -0700
    Last State:     Terminated
      Reason:       OOMKilled
      Exit Code:    137
      Started:      Fri, 07 Jul 2019 12:54:30 -0700
      Finished:     Fri, 07 Jul 2019 12:54:33 -0700
    Ready:          False
    Restart Count:  5
```

```
Conditions:
  Type      Status
  Ready     False
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  42s   default-scheduler  Successfully assigned simmemleak-hra99 
  Normal  Pulled     41s   kubelet            Container image "saadali/simmemleak:lat
  Normal  Created    41s   kubelet            Created container simmemleak
  Normal  Started    40s   kubelet            Started container simmemleak
  Normal  Killing    32s   kubelet            Killing container with id ead3fb35-5cf5
```

In the preceding example, the `Restart Count: 5` indicates that the `simmemleak` container in the Pod was terminated and restarted five times (so far). The `OOMKilled` reason shows that the container tried to use more memory than its limit.

Your next step might be to check the application code for a memory leak. If you find that the application is behaving how you expect, consider setting a higher memory limit (and possibly request) for that container.

What's next

- Get hands-on experience assigning Memory resources to containers and Pods.
- Get hands-on experience assigning CPU resources to containers and Pods.
- Read how the API reference defines a container and its resource requirements
- Read more about the local ephemeral storage
- Read more about the kube-scheduler configuration reference (v1)
- Read more about Quality of Service classes for Pods
- Read more about Extended Resource allocation by DRA

4 - Organizing Cluster Access Using kubeconfig Files

Use kubeconfig files to organize information about clusters, users, namespaces, and authentication mechanisms. The `kubectl` command-line tool uses kubeconfig files to find the information it needs to choose a cluster and communicate with the API server of a cluster.

> **Note:**
> A file that is used to configure access to clusters is called a *kubeconfig file*. This is a generic way of referring to configuration files. It does not mean that there is a file named `kubeconfig`.

> **Warning:**
> Only use kubeconfig files from trusted sources. Using a specially-crafted kubeconfig file could result in malicious code execution or file exposure. If you must use an untrusted kubeconfig file, inspect it carefully first, much as you would a shell script.

By default, `kubectl` looks for a file named `config` in the `$HOME/.kube` directory. You can specify other kubeconfig files by setting the `KUBECONFIG` environment variable or by setting the `--kubeconfig` flag.

For step-by-step instructions on creating and specifying kubeconfig files, see Configure Access to Multiple Clusters.

Supporting multiple clusters, users, and authentication mechanisms

Suppose you have several clusters, and your users and components authenticate in a variety of ways. For example:

- A running kubelet might authenticate using certificates.
- A user might authenticate using tokens.
- Administrators might have sets of certificates that they provide to individual users.

With kubeconfig files, you can organize your clusters, users, and namespaces. You can also define contexts to quickly and easily switch between clusters and namespaces.

Context

A *context* element in a kubeconfig file is used to group access parameters under a convenient name. Each context has three parameters: cluster, namespace, and user. By default, the `kubectl` command-line tool uses parameters from the *current context* to communicate with the cluster.

To choose the current context:

```
kubectl config use-context
```

The KUBECONFIG environment variable

The `KUBECONFIG` environment variable holds a list of kubeconfig files. For Linux and Mac, the list is colon-delimited. For Windows, the list is semicolon-delimited. The `KUBECONFIG` environment variable is not required. If the `KUBECONFIG` environment variable doesn't exist, `kubectl` uses the default kubeconfig file, `$HOME/.kube/config`.

If the `KUBECONFIG` environment variable does exist, `kubectl` uses an effective configuration that is the result of merging the files listed in the `KUBECONFIG` environment variable.

Merging kubeconfig files

To see your configuration, enter this command:

```
kubectl config view
```

As described previously, the output might be from a single kubeconfig file, or it might be the result of merging several kubeconfig files.

Here are the rules that `kubectl` uses when it merges kubeconfig files:

1. If the `--kubeconfig` flag is set, use only the specified file. Do not merge. Only one instance of this flag is allowed.

   Otherwise, if the `KUBECONFIG` environment variable is set, use it as a list of files that should be merged. Merge the files listed in the `KUBECONFIG` environment variable according to these rules:

   - Ignore empty filenames.
   - Produce errors for files with content that cannot be deserialized.
   - The first file to set a particular value or map key wins.
   - Never change the value or map key. Example: Preserve the context of the first file to set `current-context`. Example: If two files specify a `red-user`, use only values from the first file's `red-user`. Even if the second file has non-conflicting entries under `red-user`, discard them.

   For an example of setting the `KUBECONFIG` environment variable, see Setting the KUBECONFIG environment variable.

   Otherwise, use the default kubeconfig file, `$HOME/.kube/config`, with no merging.

2. Determine the context to use based on the first hit in this chain:

   1. Use the `--context` command-line flag if it exists.
   2. Use the `current-context` from the merged kubeconfig files.

   An empty context is allowed at this point.

3. Determine the cluster and user. At this point, there might or might not be a context. Determine the cluster and user based on the first hit in this chain, which is run twice: once for user and once for cluster:

   1. Use a command-line flag if it exists: `--user` or `--cluster`.
   2. If the context is non-empty, take the user or cluster from the context.

   The user and cluster can be empty at this point.

4. Determine the actual cluster information to use. At this point, there might or might not be cluster information. Build each piece of the cluster information based on this chain; the first hit wins:

   1. Use command line flags if they exist: `--server`, `--certificate-authority`, `--insecure-skip-tls-verify`.
   2. If any cluster information attributes exist from the merged kubeconfig files, use them.
   3. If there is no server location, fail.

5. Determine the actual user information to use. Build user information using the same rules as cluster information, except allow only one authentication technique per user:
   1. Use command line flags if they exist: `--client-certificate`, `--client-key`, `--username`, `--password`, `--token`.
   2. Use the `user` fields from the merged kubeconfig files.
   3. If there are two conflicting techniques, fail.
6. For any information still missing, use default values and potentially prompt for authentication information.

File references

File and path references in a kubeconfig file are relative to the location of the kubeconfig file. File references on the command line are relative to the current working directory. In `$HOME/.kube/config`, relative paths are stored relatively, and absolute paths are stored absolutely.

Proxy

You can configure `kubectl` to use a proxy per cluster using `proxy-url` in your kubeconfig file, like this:

```
apiVersion: v1
kind: Config

clusters:
- cluster:
    proxy-url: http://proxy.example.org:3128
    server: https://k8s.example.org/k8s/clusters/c-xxyyzz
  name: development

users:
- name: developer

contexts:
- context:
  name: development
```

What's next

- Configure Access to Multiple Clusters
- `kubectl config`

5 - Resource Management for Windows nodes

This page outlines the differences in how resources are managed between Linux and Windows.

On Linux nodes, cgroups are used as a pod boundary for resource control. Containers are created within that boundary for network, process and file system isolation. The Linux cgroup APIs can be used to gather CPU, I/O, and memory use statistics.

In contrast, Windows uses a *job object* per container with a system namespace filter to contain all processes in a container and provide logical isolation from the host. (Job objects are a Windows process isolation mechanism and are different from what Kubernetes refers to as a Job).

There is no way to run a Windows container without the namespace filtering in place. This means that system privileges cannot be asserted in the context of the host, and thus privileged containers are not available on Windows. Containers cannot assume an identity from the host because the Security Account Manager (SAM) is separate.

Memory management

Windows does not have an out-of-memory process killer as Linux does. Windows always treats all user-mode memory allocations as virtual, and pagefiles are mandatory.

Windows nodes do not overcommit memory for processes. The net effect is that Windows won't reach out of memory conditions the same way Linux does, and processes page to disk instead of being subject to out of memory (OOM) termination. If memory is over-provisioned and all physical memory is exhausted, then paging can slow down performance.

CPU management

Windows can limit the amount of CPU time allocated for different processes but cannot guarantee a minimum amount of CPU time.

On Windows, the kubelet supports a command-line flag to set the scheduling priority of the kubelet process: `--windows-priorityclass` . This flag allows the kubelet process to get more CPU time slices when compared to other processes running on the Windows host.

More information on the allowable values and their meaning is available at Windows Priority Classes. To ensure that running Pods do not starve the kubelet of CPU cycles, set this flag to `ABOVE_NORMAL_PRIORITY_CLASS` or above.

Resource reservation

To account for memory and CPU used by the operating system, the container runtime, and by Kubernetes host processes such as the kubelet, you can (and should) reserve memory and CPU resources with the `--kube-reserved` and/or `--system-reserved` kubelet flags. On Windows these values are only used to calculate the node's allocatable resources.

> **Caution:**
>
> As you deploy workloads, set resource memory and CPU limits on containers. This also subtracts from `NodeAllocatable` and helps the cluster-wide scheduler in determining which pods to place on which nodes.
>
> Scheduling pods without limits may over-provision the Windows nodes and in extreme cases can cause the nodes to become unhealthy.

On Windows, a good practice is to reserve at least 2GiB of memory.

To determine how much CPU to reserve, identify the maximum pod density for each node and monitor the CPU usage of the system services running there, then choose a value that meets your workload needs.